

[/agent](#)

[/feedback](#)

[/proposal](#)

THREE DELIVERABLES

One agent, one eval, one proposal. Three sections, one site.

The agent

Luke. 27 tools, 169 turns last month from five daily-active human users (a personal-use agent at production scale, not an enterprise load). What it does, how I instrumented it, and the trace that started this.



02

span.feedback

Field notes from instrumenting

Specific moments where Phoenix and AX worked, and where they didn't. An editorial essay rather than a feature comparison table.



03

span.proposal

Tool Bypass: an eval primitive

A failure-class taxonomy, a precise definition, real audit data over 169 turns, and a roadmap that names what would invalidate it.



IF YOU HAVE NINETY SECONDS

Failure class:

tool bypass

Detection method:

regex first, LLM-as-judge planned

Sample:

169 production turns

30 days, post v50 defense

Result:

7 real / 4 false positives

Fork status:

private, matched to your patterns

not a PR

Existing Phoenix evals catch hallucination and wrong-tool-selection. They don't catch an agent that asserts an action without firing the tool. I named the class *tool bypass* and wrote an eval for it that matches Phoenix's existing module patterns.

The eval ran against **169 real production turns** from one of my agents. It caught **seven real phantom-action bugs** and surfaced **four false positives**. The false positives are useful: they argue for an LLM-as-judge follow-up rather than a regex-only approach, and they're documented row-by-row in the proposal section.

The proposal commits to an **Alyx runtime decision** in the next thirty days (latency budget, routing surface, build vs configure), names the dependency on handoff and memory primitives that come after, and treats *where on the AX surface this primitive lives* as a question to settle with your eval team before I'd commit to a ship date.

Bryan Plaza 2026-05-01

BSPlaza@outlook.com ↓ proposal.pdf
github.com/BSPLAZA ↗ Luke public repo
linkedin.com/in/BryanIA0 ↗ Phoenix fork